Capstone Weekly Progress Report 3

ChalkStream (Group #62)

[Week of Nov 10, 2025]

Supervisor: Safaa Bedawi

Group Members: Cameron Bodkin-Dick, Eric Cui, Sandy Alzabadani

1. Overview of Current Week

This weekly report is for the weekend between the meeting for week 2 and week 3, Nov 7th - Nov 10th, so it is slightly more compact, but we would like to remain consistent with the progress reports occurring weekly. Cameron and Eric are on integrating and finalizing our preprocessing and segmentation tools, as well as getting some performance tests on the segmentation when using his NVIDIA GeForce graphics card. Sandy has been setting up everything we will need for the training environment for TrOCR with the LectureVideoDB dataset.

2. Milestone Progress Summary

Component	Description	Current Status % Complete	Remarks / Issues
3.1 Frame Preprocessing	Isolating the board from the rest of the wall, rectifying any angular distortion, and handling dynamic obstructions (instructor walking across the board)	70%	Problems with board isolation as our current scripts have trouble recognizing the board
3.2 Content Segmentation	Determine candidate areas of written content, and assign text, equation, and diagram affinity	85%	Running the PaddleOCR mobile model on a GPU has some unexpected dependencies.
3.3. Handwriting OCR	Setting up the testing & training environment for the OCR and prepping datasets.	50%	The environment for training the OCR is set up and ready to start.

3. Individual Member Contributions

<u>Cameron Bodkin-Dick</u>

This Week:

This weekend, Eric and I took a couple of hours to discuss a couple of ongoing topics. First, the board isolation/zoom feature for rectifying any angular distortion and cropping irrelevant parts of the image (wall, floor). We worked on an old prototype module that I chalked out early in the project's life, however, it's apparent that there are a lot of rules that might go into defining the edges of a board, based on whether the board is fully in frame or partially (long blackboard in some lecture halls), or wether there might be existing content on the board when the lecture initially starts. These issues don't completely stop us from being able to make a first guess on the board edges, but we'll likely want to allow the instructor to verify and alter the corner definitions if needed. The biggest considerations are that the corners must be finalized before anything is detected, and remain unchanged for that board "session".

Additionally, Eric and I attempted to set up comparative performance tests between CPU and GPU inference using an Eric CUDA-compatible NVIDIA graphics card, but it's apparent that I missed some configuration and download prerequisites/dependencies, so we weren't able to get this setup and running quite yet this weekend.

Next Week:
Troubleshoot and resolve the GPU-specific dependencies to enable to CPU vs. GPU performance tests (the paddle documentation states around a 400% increase in inference time, but I'd like to confirm this on our hardware).

The goal for week 4 is to have a lightweight orchestration layer that can demo the Preprocessing and Segmentation module features to get an initial feel for the system and check the consistency and usability. Also, reviewing the training script made by Sandy!

Eric Cui

This Week:
Had a private session with Cameron to collaborate on some topics, which did not go too well. We started off with the discussion of board isolation. I was struggling with the script as I was not able to get the script to isolate the board with any success, and Cameron mentioned he had some success while doing research. However, after extensive testing the result isn't as promising. What we did next was use my laptop's GPU to run Cameron's paddleocr script for a performance test. However, we ran into problems with setting up vscode for running this script.

Next Week:
I will create a placeholder script for board isolation and integrate the preprocessing

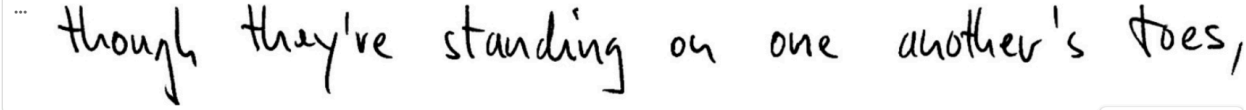
Sandy Alzabadani

This week: Ran the pretrained TrOCR Handwritten model on a small dataset to test the accuracy of the model. Generated a prototype for the Handwritten TrOCR on some IAM dataset and HuggyFace Lecture images dataset. Initially the pretrained model generated gibberish text from the lecture images dataset. This is because the images contained multiple lines of text and TrOCR only reads single lines of text. I tried cropping a lecture image to create multiple images containing single lines of text. I uploaded the cropped images on google colab and wrote a script to display the image and pass it through OCR to extract text. I was able to get an output that is very close to the text in the cropped images. IAM images produced more accurate texts than lecture images as the IAM images are clearer. Fortunately, I found cropped versions of the lecture images in the dataset folder and a text file containing the labels for all the images. Wrote a python script to fine tune the pretrained model with the cropped images and the corresponding labels and train the model accordingly. Wrote another python script to be used after the

training process is done to compare the original pretrained TrOCR with the version that we are training and compare the outputs. I ran the training model until it ran successfully with no bugs.

```
def show_image(pathStr):
    img = Image.open(pathStr).convert("RGB")
    display(img)
    return img

# Step 2: Show the image you want
img = show_image("p03-040-02.jpg")
```

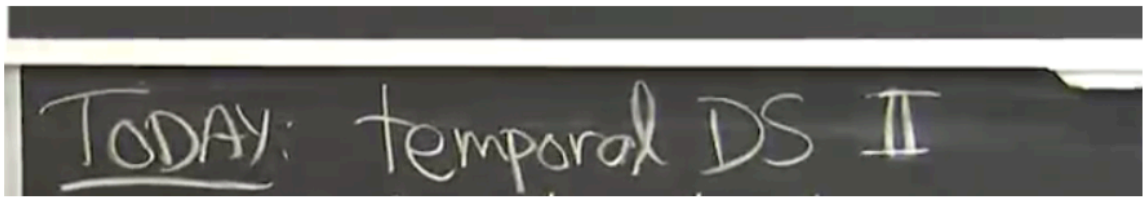



```
text = ocr_image(img)
print("OCR Result:\n", text)
```

```
OCR Result:
 though they're standing on one another's toes ,
```

Using IAM dataset (pretrained TrOCR)

```
# Display cropped image
display(cropped_img)
```




```
text = ocr_image(cropped_img)
print("OCR Result:\n", text)
```

```
OCR Result:
 TODAY : temporary DS II #
```

Using Lecture Images (pretrained TrOCR)

Next Week: Get the fine tuning script reviewed by team members and then train the model (run).

Challenges:

Faced many challenges. The first challenge was trying to find a way to crop the images of text so they could be detected by TrOCR and finally I was able to find a nested folder containing the cropped images. The second challenge is that there aren't separate text files for each text image, they are all grouped together in one labels text file along with the image paths. Had to write a function to separate the texts from the paths in the single labels text file so that it is compared with the text from the images. Another major challenge, was after setting up the training environment to fine tune the TrOCR, I ran through so many errors while running it. Had to use pyTorch instead or Tensorflow, fix the paths of data file and labels file, install more dependencies and other bugs.